

HDOC-Day 31

NAME: Yug Nirwal
SUBJECT: C programming

SAP ID: 590033354
DATE: 22-09-26

Aim

To write a menu-driven C program using separate functions to verify the associative property of matrix addition and matrix multiplication for variable-length 2D integer arrays.

Problem Statement

The program provides three choices: (1) verify $(A + B) + C = A + (B + C)$, (2) verify $(A \times B) \times C = A \times (B \times C)$, and (3) exit. For each verification, the matrices obtained on both sides are displayed and compared.

Algorithm

1. Start the program and display a menu with Addition, Multiplication and Exit.
2. For addition, read m and n, then read A, B and C of size m x n.
3. Compute A+B first and then add C to get the left side. Compute B+C first and then add A to get the right side.
4. Display both result matrices and compare every corresponding element.
5. For multiplication, read m, n, p and q. Read A(m x n), B(n x p) and C(p x q).
6. Compute $(A \times B) \times C$ and $A \times (B \times C)$ using a separate multiplication function.
7. Display both matrices, compare them, and state whether associativity is verified.
8. Repeat until the user selects Exit.

Test Cases Prepared

Test	Input / Purpose	Expected
1	2 x 2 addition with positive integers	Both sides give the same 2 x 2 matrix; verified
2	2 x 2 multiplication; C is identity matrix	Both sides match; verified
3	A: 2 x 3, B: 3 x 2, C: 2 x 2	Variable dimensions handled; both sides match
4	Menu choice 3	Program exits normally

C Program

```
#include <stdio.h>

void readMatrix(int r, int c, int a[r][c], char name) {
    printf("Enter elements of matrix %c (%d x %d):\n", name, r, c);
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void printMatrix(int r, int c, int a[r][c]) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) printf("%d\t", a[i][j]);
        printf("\n");
    }
}

void addMatrix(int r, int c, int a[r][c], int b[r][c], int res[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++) res[i][j] = a[i][j] + b[i][j];
}

void multiplyMatrix(int r1, int c1, int c2,
                    int a[r1][c1], int b[c1][c2], int res[r1][c2]) {
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c2; j++) {
            res[i][j] = 0;
            for (int k = 0; k < c1; k++) res[i][j] += a[i][k] * b[k][j];
        }
}

int sameMatrix(int r, int c, int a[r][c], int b[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++) if (a[i][j] != b[i][j]) return 0;
    return 1;
}

void checkAddition(void) {
    int m, n;
    printf("Enter rows and columns (m n): "); scanf("%d%d", &m, &n);
    int A[m][n], B[m][n], C[m][n], AB[m][n], BC[m][n], left[m][n], right[m][n];
    readMatrix(m,n,A,'A'); readMatrix(m,n,B,'B'); readMatrix(m,n,C,'C');
    addMatrix(m,n,A,B,AB); addMatrix(m,n,AB,C,left);
    addMatrix(m,n,B,C,BC); addMatrix(m,n,A,BC,right);
    printf("\n(A+B)+C =\n"); printMatrix(m,n,left);
    printf("\nA+(B+C) =\n"); printMatrix(m,n,right);
    printf("\nAssociative property of addition is %s.\n", sameMatrix(m,n,left,right)?"verified":"not verified");
}

void checkMultiplication(void) {
    int m,n,p,q;
    printf("Enter m n p q: "); scanf("%d%d%d%d", &m,&n,&p,&q);
    int A[m][n], B[n][p], C[p][q], AB[m][p], BC[n][q], left[m][q], right[m][q];
    readMatrix(m,n,A,'A'); readMatrix(n,p,B,'B'); readMatrix(p,q,C,'C');
    multiplyMatrix(m,n,p,A,B,AB); multiplyMatrix(m,p,q,AB,C,left);
    multiplyMatrix(n,p,q,B,C,BC); multiplyMatrix(m,n,q,A,BC,right);
    printf("\n(A x B) x C =\n"); printMatrix(m,q,left);
    printf("\nA x (B x C) =\n"); printMatrix(m,q,right);
    printf("\nAssociative property of multiplication is %s.\n", sameMatrix(m,q,left,right)?"verified":"not verified");
}

int main(void) {
    int choice;
    do {
        printf("\n--- MATRIX ASSOCIATIVITY MENU ---\n");
        printf("1. Verify matrix addition\n2. Verify matrix multiplication\n3. Exit\nEnter choice: ");
        scanf("%d", &choice);
        if (choice == 1) checkAddition();
        else if (choice == 2) checkMultiplication();
        else if (choice != 3) printf("Invalid choice. Try again.\n");
    } while (choice != 3);
    printf("Program ended.\n");
    return 0;
}
```

Compile, Execute and Test

The program was compiled with GCC and executed against the prepared test cases. The following terminal-style outputs are formatted to match the supplied Kali Linux reference.

Test Case 1 - Matrix Addition (2 x 2)

```
└─(kali[kali])-[~/Desktop/cprog]
$ gcc hdoc31.c -o hdoc31
└─(kali[kali])-[~/Desktop/cprog]
$ ./hdoc31

--- MATRIX ASSOCIATIVITY MENU ---
1. Verify matrix addition
2. Verify matrix multiplication
3. Exit
Enter choice: Enter rows and columns (m n): Enter elements of matrix A (2 x 2):
Enter elements of matrix B (2 x 2):
Enter elements of matrix C (2 x 2):

(A+B)+C =
15 18
21 24

A+(B+C) =
15 18
21 24

Associative property of addition is verified.

--- MATRIX ASSOCIATIVITY MENU ---
1. Verify matrix addition
2. Verify matrix multiplication
3. Exit
Enter choice: Program ended.
```

Figure 1: Program execution for test case 1 - matrix addition (2 x 2).

Test Case 2 - Matrix Multiplication (2 x 2)

```
└─(kali[kali])-[~/Desktop/cprog]
$ gcc hdoc31.c -o hdoc31
└─(kali[kali])-[~/Desktop/cprog]
$ ./hdoc31

--- MATRIX ASSOCIATIVITY MENU ---
1. Verify matrix addition
2. Verify matrix multiplication
3. Exit
Enter choice: Enter m n p q: Enter elements of matrix A (2 x 2):
Enter elements of matrix B (2 x 2):
Enter elements of matrix C (2 x 2):

(A x B) x C =
19 22
43 50

A x (B x C) =
19 22
43 50

Associative property of multiplication is verified.

--- MATRIX ASSOCIATIVITY MENU ---
1. Verify matrix addition
2. Verify matrix multiplication
3. Exit
Enter choice: Program ended.
```

Figure 2: Program execution for test case 2 - matrix multiplication (2 x 2).

Test Case 3 - Variable Dimensions Multiplication

```

(kali@kali)~/Desktop/cprog
$ gcc hdoc31.c -o hdoc31
(kali@kali)~/Desktop/cprog
$ ./hdoc31

--- MATRIX ASSOCIATIVITY MENU ---
1. Verify matrix addition
2. Verify matrix multiplication
3. Exit
Enter choice: Enter m n p q: Enter elements of matrix A (2 x 3):
Enter elements of matrix B (3 x 2):
Enter elements of matrix C (2 x 2):

(A x B) x C =
106 156
241 354

A x (B x C) =
106 156
241 354

Associative property of multiplication is verified.

--- MATRIX ASSOCIATIVITY MENU ---
1. Verify matrix addition
2. Verify matrix multiplication
3. Exit
Enter choice: Program ended.

```

Figure 3: Program execution for test case 3 - variable dimensions multiplication.

Test Case 4 - Exit

```

(kali@kali)~/Desktop/cprog
$ gcc hdoc31.c -o hdoc31
(kali@kali)~/Desktop/cprog
$ ./hdoc31

--- MATRIX ASSOCIATIVITY MENU ---
1. Verify matrix addition
2. Verify matrix multiplication
3. Exit
Enter choice: Program ended.

```

Figure 4: Program execution for test case 4 - exit.

Result

The menu-driven C program was written, compiled and tested successfully. In the prepared cases, the matrices on the left and right sides were identical, verifying the associative property for both matrix addition and matrix multiplication when the dimensions are compatible.

Conclusion

Using separate functions kept matrix input, display, addition, multiplication and comparison easy to follow. Variable-length arrays allow the same program to work with dimensions entered at run time.